Raft, Slowly

Lecture notes for a distributed systems reading group

Distributed Systems Reading Group, week 6

Why consensus is hard

A replicated service keeps copies of the same data on several machines so that it survives when one of them fails. The hard part is not copying data; it is agreeing on the order of changes when messages can be delayed, duplicated or lost, and when any machine can stop without warning.

Consensus algorithms solve this by electing one machine to decide the order and by refusing to treat a change as final until a majority of machines have stored it. A majority matters because any two majorities overlap in at least one machine, so knowledge of a committed change can never be lost entirely.

Leader election

Raft divides time into terms, numbered upward. Each term begins with an election. A follower that hears nothing from a leader for a randomized timeout becomes a candidate, increases the term, votes for itself and asks the others for their votes.

A server grants at most one vote per term, and only to a candidate whose log is at least as up to date as its own. The candidate that collects a majority becomes leader and starts sending heartbeats to stop new elections.

Randomized timeouts are the quiet trick here. Because followers time out at different moments, one candidate usually starts first and wins before the others wake up, so split votes are rare and short.

Log replication

The leader appends each client request to its log and sends the new entry to followers. When a majority have stored it, the entry is committed, the leader applies it to its state machine, and later messages tell followers to apply it too.

Each entry records the term in which it was created. Followers check that the entry before the new one matches what the leader has; if it does not, they reject it, and the leader steps back until their logs agree. This consistency check is what keeps logs identical without any separate repair process.

Questions for discussion

Why must a candidate have an up-to-date log to win, and what could go wrong if this rule were dropped?

What happens to requests the old leader accepted but never committed when it loses leadership?

How would you explain the overlap of majorities to someone without a computer science background?